

Majorit: x | Post | L: x | System x | NGSP x | CPLAB x | Maximu x | java on: x | Online x | + | Ask Gemini | - | X

hackerrank.com/contests/2026-competitive-coding-lab1/challenges/maximum-subarray-sum-5/problem?isFullScreen=true

HackerRank | All Contests > 2026 Competitive Programming CSEA SEM-1 | Maximum sub-array-sum | Exit Full Screen View

Problem

Sample Input 3

6
-1 2 3 4 5 -15

Sample Output 3

14

Sample Input 4

7
-3 4 -1 2 1 -4 3

Sample Output 4

6

Sample Input 5

Submissions

Leaderboard

Change Theme Language Java 7

```
7 public class Solution {
9     public static int maxSubArray(int[] arr) {
10         int maxSoFar = Integer.MIN_VALUE;
11         int maxEnd = 0;
12
13         for (int i = 0; i < arr.length; i++) {
14             maxEnd += arr[i];
15
16             if (maxSoFar < maxEnd) {
17                 maxSoFar = maxEnd;
18             }
19
20             if (maxEnd < 0) {
21                 maxEnd = 0;
22             }
23         }
24         return maxSoFar;
25     }
26 }
```

Line: 24 Col: 1

Upload Code as File ☐ Test against custom input Run Code Submit Code

Type here to search

31°C 22:40 12-07-2026

Problem

Submissions

Leaderboard

The merged array should also be sorted in non-decreasing order.

Complete the function `mergeSortedArrays` that takes the following parameters:

`int arr1[n]`: the first sorted array `int arr2[m]`: the second sorted array

Returns `int[]`: the merged sorted array

Input Format

The first line contains an integer `n`, the size of the first array. The second line contains `n` space-separated integers representing the first sorted array. The third line contains an integer `m`, the size of the second array. The fourth line contains `m` space-separated integers representing the second sorted array.

Constraints

$1 \leq n, m \leq 10^5$ $-10^9 \leq arr1[i], arr2[i] \leq 10^9$ Both arrays are sorted in non-decreasing order.

Output Format

Print the merged sorted array as space-separated integers.

Change Theme

Language

Java 8

```
4 public class Solution {
5     public static int[] merge(int[] arr1, int[] arr2) {
13         while (i < n && j < m) {
14             if (arr1[i] <= arr2[j]) {
15                 result[k++] = arr1[i++];
16             } else {
17                 result[k++] = arr2[j++];
18             }
19         }
20
21         while (i < n) {
22             result[k++] = arr1[i++];
23         }
24
25         while (j < m) {
26             result[k++] = arr2[j++];
27         }
28     }
29 }
```

Line: 30 Col: 6

Upload Code as File

Test against custom input

Run Code

Submit Code

Majorit: x | Post | L x | System x | NGSP x | CPLAB x | Majorit: x | java on: x | Online x | + | Ask Gemini | - | X

hackerrank.com/contests/2026-competitive-coding-lab1/challenges/majority-element-82-1/problem?isFullScreen=true

HackerRank | All Contests > 2026 Competitive Programming CSEA SEM-1 | Majority Element 82 | Exit Full Screen View

Problem

Given an integer array of size n , find the majority element. A majority element is an element that appears more than $\lfloor n/2 \rfloor$ times.

If a majority element exists, print it. Otherwise, print -1. Complete the function `majorityElement` that takes the following parameter:

`int arr[n]`: an array of integers Returns `int`: the majority element if it exists; otherwise -1.

Input Format

The first line contains an integer n , the number of elements in the array.
The second line contains n space-separated integers.

Constraints

$1 \leq n \leq 10^5$ $-10^9 \leq arr[i] \leq 10^9$

Output Format

Print the majority element if it exists; otherwise, print -1.

Submissions

Leaderboard

Change Theme Language Java 15

```
4 public class Solution {
5     public static int majorityElement(int[] arr) {
6         int candidate = 0;
7         int count = 0;
8         for (int num : arr) {
9             if (count == 0) {
10                candidate = num;
11                count = 1;
12            } else if (num == candidate) {count++;
13            } else {count--;
14            }
15        } count = 0;
16        for (int num : arr) {
17            if (num == candidate) {
18                count++;
19            }
20        } if (count > arr.length / 2) {
21            return candidate;
```

Line: 15 Col: 10

Upload Code as File ☐ Test against custom input Run Code Submit Code

Type here to search | 31°C | 22:36 12-07-2026

Majorit: x | Post | Li x | System x | NGSP x | CPLAB x | Maxim x | java on: x | Online x | + | Ask Gemini | - | X

hackerrank.com/contests/2026-competitive-coding-lab1/challenges/maximum-ascending-subarray-sum-1/problem?isFullScreen=true | School

HackerRank | All Contests > 2026 Competitive Programming CSEA SEM-1 | Maximum Ascending Subarray Sum 1 | Exit Full Screen View

Problem

A subarray is ascending if every element is strictly greater than the previous element.

Return the maximum possible sum among all ascending subarrays.

Complete the function `maxAscendingSum` that takes the following parameter: `int arr[n]`: an array of positive integers. Returns `int`: the maximum ascending subarray sum.

Input Format

The first line contains an integer `n`, the number of elements in the array. The second line contains `n` space-separated integers.

Constraints

$1 \leq n \leq 10^5$ $1 \leq arr[i] \leq 10^4$

Output Format

Print a single integer representing the maximum ascending subarray sum.

Sample Input 0

Change Theme Language Java 15

```
4 public class Solution {
5
6     public static int asendSum(int[] arr) {
7         int maxSum = arr[0];
8         int currentSum = arr[0];
9
10        for (int i = 1; i < arr.length; i++) {
11            if (arr[i] > arr[i - 1]) {
12                currentSum += arr[i];
13            } else {
14                currentSum = arr[i];
15            }
16
17            if (currentSum > maxSum) {
18                maxSum = currentSum;
19            }
20        }
21        return maxSum;
22    }
```

Line: 19 Col: 14

Upload Code as File ☐ Test against custom input Run Code Submit Code

Windows taskbar: Type here to search | 31°C | 22:34 12-07-2026